An FPGA Implementation of LSTM for ECG Processing

Panagiotis Zintzovas*, Vassilis Paliouras, and Ioannis Kouretas

Department of Electrical and Computer Engineering,
University of Patras, Patras, Greece
phy5286@ac.upatras.gr, paliuras@upatras.gr, kouretas@upatras.gr

Abstract

An FPGA implementation of a single-layer Long Short-Term Memory (LSTM) network for electrocardiogram (ECG) waveform segmentation is presented. A MATLAB reference model is first reproduced in C++ and quantization-aware training is applied to obtain an 8-bit representation suitable for hardware deployment. The quantized model is mapped to a synthesizable design using Vitis HLS and implemented on an AMD Xilinx Zynq UltraScale+ ZCU104. The derived LSTM accelerator preserves the per-class behavior of the floating-point baseline LSTM model. The mean-per-class (MPC) accuracy decreases only slightly (61.28% to 58.92%), demonstrating that fixed-point quantization maintains performance. The implemented circuit operate at 160 MHz and a latency of 46.9 ms for a 5000-sample segment. Resource utilization is 54,284 LUTs, 47,249 FFs, 410 DSP slices, and 410 BRAM blocks and demonstrates the feasibility of deploying LSTM-based ECG segmentation on resource- and power-constrained FPGA platforms for real-time applications.

1 Introduction

Electrocardiographic analysis (ECG) underpins arrhythmia detection and continuous cardiac monitoring, yet the deployment of accurate detectors on edge devices remains constrained by limited computational and energy resources. Traditional signal processing pipelines, based on hand-crafted features and thresholding methods [1], often lack robustness under noise and variability between patients. On the other hand, deep learning approaches, and in particular recurrent architectures such as Long Short-Term Memory (LSTM) networks, have shown strong capability in capturing temporal dependencies in biosignals, enabling reliable detection of clinically relevant events [2].

Field-Programmable Gate Arrays (FPGAs) provide an integrated platform for biomedical signal processing because they combine low-latency parallelism with energy efficiency appropriate for wearable and embedded systems [3]. Nevertheless, mapping recurrent networks efficiently to FPGA fabric requires careful architectural design to satisfy real-time constraints while balancing logic, memory, and power consumption. Early FPGA-based LSTM accelerators demonstrate that exploiting on-chip memory and fine-grain parallelism can deliver substantial speedups over software implementations [4]. Subsequent work extended these principles to other neural time-series tasks such as EEG, showing that compact and causal LSTM architectures can meet the latency budgets required for biomedical applications [5]. For ECG analysis, researchers have explored quantized and binarized LSTM designs to reduce multiplications and memory footprint while maintaining accuracy, thereby making real-time deployment more practical on reconfigurable devices [6]. In parallel, FPGA implementations of one-dimensional convolutional neural networks (1D-CNNs) for ECG classification have illustrated the feasibility of deep learning–based ECG analytics on embedded hardware and introduced architectural strategies that are transferable to LSTM-based solutions [7]. More recently, hybrid CNN–LSTM models have been proposed to combine local morphological feature extraction with long-range temporal modeling, further improving arrhythmia classification performance [8].

*Corresponding author.

More specifically, Liu et al. [9] proposed an FPGA-based 1D-CNN for arrhythmia classification, emphasizing compact resource utilization and low-power operation. Zhang et al. [10] introduced a binarized depthwise separable CNN on FPGA, targeting ultra-low-power deployment for portable ECG monitoring. Mangaraj et al. [11] presented a Pynq-Z2 system that integrates deep ECG inference with secure data transmission, demonstrating the feasibility of combining signal analytics with system-level features on low-cost devices. Together, these studies highlight the benefits of FPGA-based approaches to ECG analysis, ranging from classification to system-level integration, and provide a reference point for the present work.

In this paper, an FPGA implementation of an LSTM-based ECG segmentation accelerator is presented targeting the Xilinx Zynq UltraScale+ ZCU104. The design incorporates quantization-aware training, gate-level scheduling, and memory organization techniques to maintain real-time throughput with efficient utilization of logic and DSP resources. Evaluation on standard ECG datasets demonstrates that the accelerator achieves near-baseline accuracy relative to floating-point references, while providing throughput comfortably above clinical requirements.

The remainder of this paper is organized as follows. Section 2 introduces the segmentation task and the baseline LSTM model. Section 3 offers details of the FPGA implementation, including quantization and architectural optimizations. Section 4 presents experimental results and a comparison with recent FPGA designs. Finally, Section 5 concludes the paper.

2 Methodology

A Long Short-Term Memory (LSTM) unit augments a recurrent neural network with an internal *cell state* c_t and three multiplicative gates that regulate information flow. At each time step t, given input x_t and previous hidden state h_{t-1}, the gates are $i_t = \sigma(W_i x_t + U_i h_{t-1} + b_i)$ (input gate), $f_t = \sigma(W_f x_t + U_f h_{t-1} + b_f)$ (forget gate), $o_t = \sigma(W_o x_t + U_o h_{t-1} + b_o)$ (output gate), where $\sigma(\cdot)$ is the sigmoid function. The candidate update is $\tilde{c}_t = \tanh(W_c x_t + U_c h_{t-1} + b_c)$. The cell and hidden states then evolve as $c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$, $h_t = o_t \odot \tanh(c_t)$, where $\odot$ denotes elementwise multiplication. This gating mechanism allows LSTMs to preserve long-term ECG morphology (e.g., P–QRS–T cycles) while remaining sensitive to short-lived events such as arrhythmia onsets.

Segmenting an electrocardiogram (ECG) trace into P, QRS, T, and n/a regions enables downstream analysis such as rhythm classification and interval measurement. We follow the public MATLAB *Waveform Segmentation* reference [12], which labels each sample in a 1×5000 window and uses a single-layer LSTM followed by a linear head that outputs four class logits per time step. The segmentation of these regions of ECG waveforms can provide the basis for measurements useful for assessing the overall healrth of the human heart and the presence of abnormalities [13]. Manually annotating each region of the ECG signal can be a tedious and time-consuming task. Signal processing and deep learning methods potentially can help streamline and automate region-of-interest annotation.

The MATLAB rawNet baseline model takes a single ECG segment (sampled at 250 Hz) and predicts a label for every sample: n/a, P, QRS, or T. The architecture of the model is *sequenceInputLayer(1)* → *lstmLayer(200,'OutputMode','sequence')* → *fullyConnectedLayer(4)* → *softmaxLayer*. A single-direction LSTM with hidden size 200 feeding a linear head that outputs four logits per time step. During training, softmax with cross-entropy is used; at inference the predicted label is arg max over the four logits (equivalently, over softmax probabilities). The proposed hardware accelerator therefore omits softmax and applies arg max per time step. The MATLAB baseline model has been reimplemented in plain `C++` and serves as a reference model for hardware validation. Each ECG segment of length 5000 samples is processed sequentially by a single-layer LSTM followed by a linear classification head. At every time step t, the input vector $x_t \in \mathbb{R}^D$ is concatenated with the previous hidden state

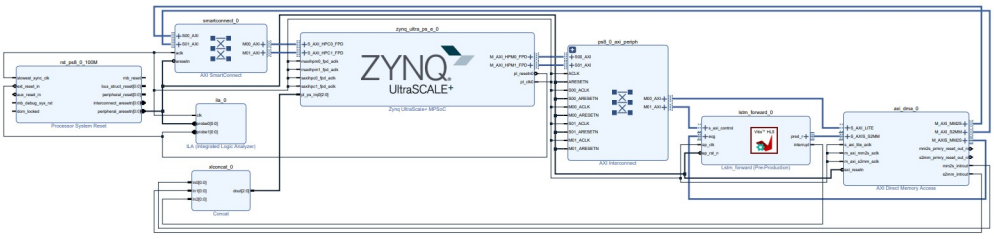

Figure 1: Top-level block diagram on ZCU104: PS (Zynq MPSoC), AXI interconnect/DMA, HLS LSTM core (IFGo), BRAM/ROM for weights and LUTs, and AXI4-MM to external memory.

$h_{t-1} \in \mathbb{R}^H$, and multiplied by the full weight matrix $W_{\text{all}} \in \mathbb{R}^{4H\times(D+H)}$ with bias $b_{\text{all}} \in \mathbb{R}^{4H}$, i.e., $z_t = W_{\text{all}} \begin{bmatrix} x_t \\ h_{t-1} \end{bmatrix} + b_{\text{all}} = \left[z_t^i \parallel z_t^f \parallel z_t^o \parallel z_t^g\right]$, where the four partitions correspond to the *input*, *forget*, *output*, and *candidate* gates (IFoG order, consistent with the MATLAB implementation). The hidden and cell states are reinitialized $(h_0, c_0) = (0, 0)$ at the start of each segment, matching the MATLAB protocol. This `C++`/FP32 implementation serves as the golden reference against which the quantized and HLS versions are verified.

3 FPGA implementation

To enable efficient deployment on FPGA, quantization-aware training (QAT) [14, 15] was applied in PyTorch FX graph mode [16], starting from the MATLAB-trained weights. QAT uses fake-quantization modules with straight-through estimators so that backpropagation flows through the rounding. We attach `MovingAverageMinMaxObserver` to activations (per-tensor symmetric) and `PerChannelMinMaxObserver` to weights (per-channel symmetric); during training these observers track ranges and update scales, and at export (`convert_fx`) the INT8 weights plus quant/dequant stubs are materialized. With symmetric quantization (zero-point z=0), the mapping is

$$q = \text{clamp}\big(\text{round}(x/s),\, q_{\min}, q_{\max}\big), \qquad x \approx s\, q,$$

using $[-127, 127]$ for per-channel weights and $[-128, 127]$ for per-tensor activations. Accumulators and biases remain 32-bit (or wider) to avoid overflow. To address the strong imbalance toward the "n/a" class, mini-batches were drawn with PyTorch's `WeightedRandomSampler` using class weights $w_c \propto 1/(f_c + \epsilon)$; evaluation used the natural distribution and reported per-class accuracy and mean-per-class (MPC) [17]. A short calibration pass on held-out segments stabilized activation ranges and informed the clamp domains of the LUTs for σ and tanh (e.g., $[-8, 8]$ with linear interpolation).

The PyTorch/QAT implementation followed the IFGo gate convention, $z_t = [z_t^i \parallel z_t^f \parallel z_t^g \parallel z_t^o]$, where the partitions correspond to the input, forget, candidate, and output gates. This order is mathematically equivalent to the IFoG layout of MATLAB, differing only by a row permutation of W_{all} and b_{all}. At export, the rows were permuted explicitly so that all downstream artifacts (headers, lookup tables, and HLS kernels) observed a consistent convention. The exported package included fused C headers for the LSTM and classification head (`W_all_fixed.h`, `B_all_fixed.h`, `W_fc_fixed.h`, `b_fc_fixed.h`) as well as the activation LUTs. The LSTM forward pass was realized as a synthesizable core in Vitis HLS and integrated into the programmable logic of the Zynq UltraScale+ ZCU104 via AXI interfaces. The top-level block design instantiates the HLS kernel, AXI interconnect and DMA engines, on-chip BRAM/ROM for weights and lookup tables, and AXI4-MM ports for streaming data between external DDR and the fabric (Fig. 1).

Control registers for segment length, buffer addresses, and execution count are exposed through an AXI-Lite interface, enabling software on the ARM cores to configure the accelerator.

ECG samples arrive over AXI as 8-bit integers, reflecting the quantization used at training time. On chip, a separation between storage and compute types is enforced. Weights, biases, and nonlinearity lookup tables are stored in BRAM as `ap_fixed<16,6>` (denoted `state_t`), which strikes a balance between compact memory footprint and sufficient fractional precision. During computation, inputs and recurrent states are promoted to `ap_fixed<32,24>` (`acc_t`) for multiply–accumulate operations, ensuring that partial sums do not saturate. Gate outputs and logits are cast back to `state_t` for storage, while the cell state employs a wider format `ap_fixed<26,14>` (`cell_t`) to maintain precision over long temporal dependencies. Before evaluation by the tanh lookup table, c_t is cast to `acc_t`. The denominator of the softmax is accumulated in `ap_fixed<24,6>` (`sum_t`) to avoid overflow when summing across classes.

The kernel follows the IFGo gate order, This ordering allows $\sigma(z_t^i)$, $\sigma(z_t^f)$, and $\tanh(z_t^g)$ to be computed first, enabling $c_t = f_t \odot c_{t-1} + i_t \odot g_t$ to be formed early. While the $\tanh(c_t)$ lookup is in progress, $\sigma(z_t^o)$ can be evaluated, allowing the update $h_t = o_t \odot \tanh(c_t)$ to complete without stalls. This scheduling overlap shortens the critical path compared with the canonical IFoG ordering and reduces the latency of each iteration. The time-step loop was fully pipelined with an initiation interval (II) of one. In practice, this means that after the pipeline is filled, a new ECG sample is accepted and processed every clock cycle. Achieving II=1 required partitioning weight matrices across multiple BRAM banks to allow parallel reads, binding multiply–accumulate operations to dedicated DSP48 slices, and unrolling gate computations to eliminate loop-carried dependencies. Under this schedule, operations at time t (affine transform, gate activations, state update, classification head) overlap with those of time $t+1$, so throughput is limited only by the clock frequency. For the ZCU104 device, this corresponds to one output label per cycle, i.e., 100 MSamples/s at 100 MHz. The sustained end-to-end throughput measured on hardware was 6.7×10^4 samples/s when including DMA transfers and software orchestration.

Weights, biases, and lookup tables are compiled into ROMs mapped to BRAM/URAM blocks. ECG samples are fetched from external memory in bursts and streamed to the core through AXI4-MM. Predictions are emitted one per cycle and written back via DMA. Hidden and cell states are retained in small ping–pong buffers inside the kernel to avoid memory traffic. Dataflow pragmas are applied at the top level so that data movement and computation overlap. As a result, the accelerator maintains steady throughput with negligible idle cycles once streaming begins.

4 Results

Evaluation was performed on the held-out test split used by the MATLAB reference, with each ECG segment of length 1×5000 samples. At every time step the network produced four logits, and the predicted label was obtained by applying $\arg\max$. Because the dataset is dominated by the "n/a" class, overall accuracy is not considered representative. Instead, per-class accuracy for the *n/a*, *P*, *QRS*, and *T* classes is reported together with the mean-per-class (MPC). Table 1 summarizes the per-class performance of the MATLAB FP32 baseline, the plain C++ reproduction, the INT8 model obtained with quantization-aware training (QAT), and the FPGA implementation. The `C++` path reproduces MATLAB exactly, confirming numerical correctness of the oracle. The INT8 QAT model maintains baseline accuracy, and the FPGA implementation closely follows it. Relative to MATLAB, the FPGA shows the largest deviation on the *n/a* class (-9.2 pp), accompanied by small decreases in *P* and *QRS* and a modest increase in *T*. These shifts represent a redistribution away from the majority class, which results in only a 2.4 pp reduction in MPC, from 61.3% to 58.9%.

Table 2 compares the proposed design with recent FPGA accelerators for ECG or related

Table 1: Per-class accuracy and mean-per-class (MPC) on the test set. Bottom row reports the difference in percentage points (pp) between the FPGA and MATLAB baselines.

Model	n/a (%)	P (%)	QRS (%)	T (%)	MPC (%)
MATLAB FP32	86.9	39.4	60.9	57.9	61.28
C++ FP32	86.9	39.4	60.9	57.9	61.28
INT8 (QAT)	86.68	40.25	61.00	58.38	61.58
FPGA (INT8, HLS)	77.71	38.25	59.04	60.67	58.92
FPGA–MATLAB (pp)	−9.19	−1.15	−1.86	+2.77	−2.36

Table 2: Recent FPGA ECG accelerators compared with this work.

Work	**FPGA**	$\mathbf{F_{max}}$	**LUT**	**DSP**	**BRAM**	**Power**	**Model**
This work (2025)	ZCU104	160 MHz	54k	410	410	3.369 W	LSTM
Liu et al. [9]	Zynq-7000	50 MHz	13.7k	9	5.5	1.78 W	1D-CNN
Zhang et al. [10]	DE1-SoC	0.1 MHz	3.8k*	0	—	20 μW	bDSCNN
Mangaraj et al. [11]	Pynq-Z2	2.5 MHz	5.3k	—	1k LUTRAM	—	1D-CNN

*Reported as LUT+REG.

biosignal tasks. Liu et al. [9] implemented a 1D-CNN on a Zynq-7000, achieving low DSP usage (9 slices) and 1.78 W at 50 MHz, but with lower throughput and targeting arrhythmia classification rather than segmentation. Zhang et al. [10] presented a binarized DSCNN on a Cyclone V device, achieving ultra-low power (20 μW) at 100 kHz with no DSP consumption, but operating far below real-time rates. Mangaraj et al. [11] demonstrated a system-level design with security features on a Pynq-Z2, reporting only 5,269 LUTs but limited to 2.5 MHz with substantial LUTRAM use. These results illustrate different points in the design space. Prior CNN-based implementations emphasize either low DSP usage and modest power (Liu), ultra-low power operation at very low frequency [10], or integration on low-cost platforms with additional system functionality [11]. In contrast, the proposed LSTM accelerator demonstrates that near-baseline accuracy and real-time ECG segmentation can be achieved on a mid-range SoC FPGA with modest resource utilization and throughput comfortably above clinical requirements. *The proposed architecture can achieve substantial power reduction by operating at a lower clock frequency, provides throughput-rate* vs. *power dissipation tradeoffs, allowing processing of longer input sequences, and/or supporting higher sampling rates.*

5 Conclusion

This paper presented an FPGA implementation of a single-layer LSTM for ECG waveform segmentation. Starting from a MATLAB reference, the design was reproduced in C++, quantized to 8-bit using quantization-aware training, and deployed as a synthesizable HLS core on a Xilinx Zynq UltraScale+ ZCU104. The accelerator preserved the reference model's per-class behavior with only minor shifts, achieving a mean-per-class accuracy of 58.9% versus 61.3% for the baseline. Post-implementation results showed timing closure at 160 MHz with an initiation interval of one, yielding 46.9 ms latency per 1×5000 segment (106.5 k samples/s). Resource use was modest (54k LUTs, 47k FFs, 410 DSPs, 410 BRAMs), demonstrating that recurrent networks can be deployed efficiently on mid-range SoC FPGAs for real-time ECG analysis. The results confirm the practicality of combining quantization-aware training for recurrent models such as LSTM, with HLS design to enable clinically relevant inference on edge platforms.

References

[1] Wasimuddin, Muhammad and Elleithy, Khaled and Abuzneid, Abdel-Shakour and Faezipour, Miad and Abuzaghleh, Omar, "Stages-Based ECG Signal Analysis From Traditional Signal Processing to Machine Learning Approaches: A Survey," *IEEE Access*, vol. 8, pp. 177782–177803, 2020.

[2] Z. Wu and C. Guo, "Deep learning and electrocardiography: systematic review of current techniques in cardiovascular disease diagnosis and management," *BioMedical Engineering OnLine*, vol. 24, p. 23, Feb 2025.

[3] A. Sanaullah, C. Yang, Y. Alexeev, K. Yoshii, and M. C. Herbordt, "Real-time data analysis for medical diagnosis using FPGA-accelerated neural networks," *BMC Bioinformatics*, vol. 19, p. 490, Dec 2018.

[4] J. C. Ferreira and J. Fonseca, "An FPGA implementation of a long short-term memory neural network," in *2016 International Conference on ReConFigurable Computing and FPGAs (ReConFig)*, pp. 1–8, 2016.

[5] Z. Chen, A. Howe, H. T. Blair, and J. Cong, "FPGA-based LSTM Acceleration for Real-Time EEG Signal Processing: (Abstract Only)," FPGA '18, (New York, NY, USA), p. 288, Association for Computing Machinery, 2018.

[6] Mirsalari, Seyed Ahmad and Nazari, Najmeh and Ansarmohammadi, Seyed Ali and Sinaei, Sima and Salehi, Mostafa E. and Daneshtalab, Masoud, "ELC-ECG: Efficient LSTM Cell for ECG Classification Based on Quantized Architecture," in *2021 IEEE International Symposium on Circuits and Systems (ISCAS)*, pp. 1–5, 2021.

[7] V. B. K. L. Aruna, E. Chitra, and M. Padmaja, "Accelerating deep convolutional neural network on FPGA for ECG signal classification," *Microprocessors and Microsystems*, vol. 103, p. 104939, 2023.

[8] A. Sun, W. Hong, J. Li, and J. Mao, "An Arrhythmia Classification Model Based on a CNN-LSTM-SE Algorithm," *Sensors*, vol. 24, no. 19, 2024.

[9] Z. Liu, X. Ling, Y. Zhu, and N. Wang, "FPGA-based 1D-CNN accelerator for real-time arrhythmia classification," *Journal of Real-Time Image Processing*, vol. 22, p. 66, Feb 2025.

[10] R. Zhang, R. Zhou, Z. Zhong, H. Qi, and Y. Wang, "A Multi-Class ECG Signal Classifier Using a Binarized Depthwise Separable CNN with the Merged Convolution–Pooling Method," *Sensors*, vol. 24, no. 22, p. 7207, 2024.

[11] S. Mangaraj, J. P. Mohanty, S. Ari, A. K. Swain, and K. Mahapatra, "PACAC: PYNQ Accelerated Cardiac Arrhythmia Classifier with secure transmission- A Deep Learning based Approach," in *Proceedings of the Great Lakes Symposium on VLSI 2024*, GLSVLSI '24, (New York, NY, USA), p. 670–675, Association for Computing Machinery, 2024.

[12] MathWorks, "Waveform segmentation using deep learning." https://www.mathworks.com/help/signal/ug/waveform-segmentation-using-deep-learning.html, 2020. Accessed 2024-11-04.

[13] P. Laguna, R. Jané, and P. Caminal, "Automatic detection of wave boundaries in multilead ecg signals: Validation with the cse database," *Computers and Biomedical Research*, vol. 27, no. 1, pp. 45–60, 1994.

[14] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, and D. Kalenichenko, "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in *Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 2704–2713, 2018.

[15] R. Krishnamoorthi, "Quantizing deep convolutional networks for efficient inference: A whitepaper." https://arxiv.org/abs/1806.08342, 2018. Accessed 2024-11-04.

[16] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Kopf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala, "Pytorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems* (H. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché-Buc, E. Fox, and R. Garnett, eds.), vol. 32, Curran Associates, Inc., 2019.

[17] PyTorch, "torch.utils.data.weightedrandomsampler." https://pytorch.org/docs/stable/data.html#torch.utils.data.WeightedRandomSampler, 2024. Accessed 2024-11-04.